

Resolução do Problema da Partição

Análise Comparativa de Heurísticas

Isabela M. Andrade Júlia A. da Rocha Otávio de Oliveira Pedro B. Luccas

Departamento de Ciências da Computação
Universidade Federal de Alfenas (UNIFAL-MG)

Algoritmos e Estruturas de Dados III

- 1 O problema da partição
- 2 Definição formal
- 3 Algoritmos e pseudocódigos
- 4 Instâncias de teste
- 5 Configuração experimental
- 6 Resultados e gráficos
- 7 Discussão
- 8 Conclusão

1. O Problema da Partição

Definição

Dividir um conjunto de inteiros positivos em **dois subconjuntos** cujas somas sejam o mais próximas possível.

- Pertence à classe **NP-Completa**: não há algoritmo polinomial conhecido para a solução ótima de toda instância.
- Heurísticas e meta-heurísticas tornam-se essenciais para soluções de alta qualidade em tempo viável.

Aplicações: balanceamento de carga, escalonamento em multiprocessadores, distribuição de recursos, planejamento logístico e problemas de alocação.

2. Definição Formal

Entrada

Conjunto $A = \{a_1, a_2, \dots, a_n\}$ de inteiros positivos.

Saída

Subconjuntos disjuntos $A_1 \subseteq A$ e $A_2 = A - A_1$.

Restrições

- Cada elemento em exatamente um subconjunto;
- Nenhum elemento descartado;
- Partição completa de A .

Função objetivo

$$\min f(x) = | \text{Soma}(A_1) - \text{Soma}(A_2) |$$

O valor ótimo $f(x) = 0$ caracteriza uma **partição perfeita**.

3. Algoritmos Avaliados

Quatro abordagens em C: duas de **referência** e duas **meta-heurísticas** sob análise.

Referência (baseline)

- **Partition Puro:** todos os elementos no Grupo A.
- **Guloso (LPT):** ordena em ordem decrescente e insere cada item no subconjunto de menor soma.

Meta-heurísticas (foco)

- **Simulated Annealing:** inspirado no recozimento de metais.
- **Algoritmo Genético:** inspirado na evolução natural.

Solução: vetor binário de tamanho n ($0 \rightarrow$ Grupo A, $1 \rightarrow$ Grupo B); *fitness* = diferença absoluta entre as somas.

3.1 Pseudocódigo — Algoritmos de Referência

Partition Puro — $\mathcal{O}(n)$

```
PartitionPuro(inst):  
    // todos no Grupo A  
    sol.config  $\leftarrow$  {0,...,0}  
    sol.fitness  $\leftarrow$  soma_total  
    retorne sol
```

Guloso LPT — $\mathcal{O}(n \log n)$

```
GulosoLPT(inst):  
    itens  $\leftarrow$  ordenar_decrescente(valores)  
    somaA  $\leftarrow$  0; somaB  $\leftarrow$  0  
    para cada item em itens faca:  
        se somaA  $\leq$  somaB:  
            config[item]  $\leftarrow$  0; somaA  $\leftarrow$  somaA + item  
        senao:  
            config[item]  $\leftarrow$  1; somaB  $\leftarrow$  somaB + item  
    sol.fitness  $\leftarrow$  |somaA - somaB|  
    retorne sol
```

Fonte: Relatório, Seções 2.2 e 2.3.

3.2 Simulated Annealing (SA)

Ideia geral

Inicia com temperatura elevada (ampla exploração) e resfria progressivamente, tornando-se mais seletivo.

- **Vizinhança:** *bit-flip* de um elemento aleatório.
- **Avaliação $O(1)$:**
 $f = |2 \text{Soma}_A - \text{Soma}_{\text{total}}|$.
- **Aceitação (Metropolis):** pior solução aceita com $P = e^{-\Delta f/T}$.

Critério de parada

Temperatura mínima atingida *ou* partição perfeita encontrada.

Complexidade

$$O(I \cdot n)$$

I = níveis de temperatura

3.2 Pseudocódigo — Simulated Annealing

```
SimulatedAnnealing(inst, T0, Tf,  $\alpha$ ):  
  T  $\leftarrow$  T0  
  sol  $\leftarrow$  solucao_aleatoria(n);  melhor  $\leftarrow$  sol  
  somaA  $\leftarrow$  calcular_somaA(sol)  
  enquanto T > Tf faca:  
    repetir max_iter vezes:  
      idx  $\leftarrow$  elemento_aleatorio()  
      nova_somaA  $\leftarrow$  somaA +/- valores[idx]  // conforme o grupo atual  
      novo_fit  $\leftarrow$  |2.nova_somaA - soma_total|  
       $\Delta_f$   $\leftarrow$  novo_fit - sol.fitness  
      se  $\Delta_f$  < 0 ou random() < exp(- $\Delta_f$  / T):  
        aplicar_flip(idx); somaA  $\leftarrow$  nova_somaA; sol.fitness  $\leftarrow$  novo_fit  
        se novo_fit < melhor.fitness: melhor  $\leftarrow$  sol  
  T  $\leftarrow$  T  $\cdot$   $\alpha$   
  retorne melhor
```

Versão resumida. Fonte: Relatório, Seção 2.4.

3.3 Algoritmo Genético (AG)

Ideia geral

População de soluções evolui por seleção, cruzamento e mutação ao longo das gerações.

- **Seleção:** torneio ($k = 3$) — vence o menor *fitness*.
- **Crossover:** um ponto de corte.
- **Mutação:** *bit-flip*, taxa base $2/n$ ($\times 3$ nas últimas gerações).
- **Elitismo:** melhor indivíduo preservado.

Critério de parada

Máximo de gerações atingido *ou* solução ótima encontrada.

Complexidade

$$\mathcal{O}(G \cdot P \cdot n)$$

G = gerações, P = população

3.3 Pseudocódigo — Algoritmo Genético

```
Genetico(inst, TAM_POP, GERACOES, TX_CROSS, TX_MUT):  
  pop ← populacao_aleatoria(TAM_POP, n);  avaliar(pop)  
  melhor ← melhor_de(pop)  
  para g ← 1 ate GERACOES faca:  
    se melhor.fitness = 0: pare  
    // mutacao adaptativa (ultimas geracoes)  
    se g > 0.8·GERACOES: TXm ← 3·TX_MUT  senao: TXm ← TX_MUT  
    nova_pop[0] ← melhor_de(pop)           // elitismo  
    para i ← 1 ate TAM_POP-1 passo 2 faca:  
      p1 ← torneio(pop,3);  p2 ← torneio(pop,3)  
      (f1,f2) ← crossover_1ponto(p1, p2, TX_CROSS)  
      mutacao_bitflip(f1, TXm);  mutacao_bitflip(f2, TXm)  
      avaliar(f1);  avaliar(f2)  
      nova_pop[i] ← f1;  nova_pop[i+1] ← f2  
    pop ← nova_pop;  melhor ← atualizar_melhor(pop, melhor)  
  retorne melhor
```

Versão resumida. Fonte: Relatório, Seção 2.5.

3.4 Resumo das Complexidades

Algoritmo	Complexidade
Partition Puro	$\mathcal{O}(n)$
Guloso (LPT)	$\mathcal{O}(n \log n)$
Simulated Annealing	$\mathcal{O}(I \cdot n)$
Algoritmo Genético	$\mathcal{O}(G \cdot P \cdot n)$

Tabela 1. Complexidade assintótica de cada abordagem. Fonte: Relatório, Seção 2.

4. Descrição das Instâncias

Geradas automaticamente por um gerador em **linguagem C**, com **5 instâncias independentes** por tamanho.

Diversidade: variam em número de elementos, soma total, distribuição dos valores e dificuldade de balanceamento.

$N \in [100, 10000]$ • **Total: 35 instâncias.**

Tamanho (N)	Qtd.
100	5
200	5
500	5
1000	5
2500	5
5000	5
10000	5

Tabela 2. Características das instâncias. Fonte: Relatório, Seção 3.

5. Configuração Experimental

Ambiente: C (C99) • GCC • Linux • 5 execuções/instância • semente = 42.

Algoritmo Genético	
População	50 ou 100
Taxa de Crossover	0,85
Taxa de Mutação	$2/n$
Elitismo	1 indivíduo
Seleção	Torneio ($k = 3$)
N.º de Gerações	100–600

Simulated Annealing	
Temperatura Inicial	1000
Temperatura Final	0,001
Alfa (α)	0,97
Iterações por Temperatura	$\max(500, n/10)$

Tabela 3. Parâmetros das meta-heurísticas. Fonte: Relatório, Seção 4.1.

6. Resultados Obtidos

Tempos médios de execução (s)				
<i>N</i>	Partition Puro	Sim. Annealing	Alg. Genético	Guloso
100	0.000000	0.002320	0.005360	0.000000
200	0.000000	0.004480	0.011160	0.000000
500	0.000000	0.007400	0.063600	0.000040
1000	0.000000	0.007600	0.322520	0.000160
2500	0.000000	0.007240	0.644120	0.000280
5000	0.000000	0.021400	1.805360	0.000480
10000	0.000040	0.036840	7.121720	0.001080

Fitness médios (qualidade — menor é melhor; ótimo = 0)				
<i>N</i>	Partition Puro	Sim. Annealing	Alg. Genético	Guloso
100	569.049,60	6,88	7,68	29,20
200	1.168.274,20	0,92	9,56	24,60
500	3.104.440,80	1,44	4,16	29,20
1000	5.924.343,00	1,00	2,44	12,60
2500	14.773.742,80	0,40	3,12	2,40
5000	29.583.726,60	0,84	1,56	1,40
10000	59.488.785,80	0,60	1,00	1,80

Tabela 4. Médias de 5 execuções por *N* (175 testes/algorithm). Fonte: Relatório, Seção 4.2.

6.1 Gráficos — Comparação Consolidada

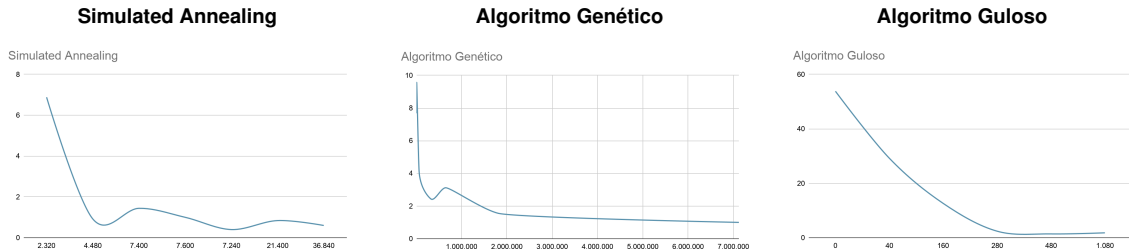


Figura 1. Curvas individuais de *fitness* de cada heurística. Fonte: Relatório, Seção 4.3.3.

7. Discussão dos Resultados

Qualidade da solução

SA produziu as melhores soluções em *todas* as instâncias (fitness $< 1,0$; ex. 0,40 em $N = 2500$). O **AG** também foi excelente, porém ligeiramente inferior ao SA.

Tempo e escalabilidade

Guloso: menor custo absoluto ($\approx 0,001$ s em $N = 10000$), competitivo em N grande.
SA: melhor escalabilidade (apenas 0,036 s no pior caso). **AG**: forte penalização temporal — de 0,005 s a mais de 7,12 s.

O **Partition Puro** cumpriu o papel de *baseline*, evidenciando o ganho das demais abordagens.

8. Conclusão

- Evidenciou-se o **compromisso entre eficiência e qualidade** em problemas NP-Completo.
- **SA** foi o mais robusto: melhor qualidade e melhor escalabilidade.
- **AG** entrega ótimas soluções, mas com alto custo temporal em instâncias grandes.
- O **Guloso** surpreende pela qualidade competitiva em N elevado a custo quase nulo.

Trabalhos futuros

Operadores genéticos mais sofisticados • diferentes esquemas de resfriamento • abordagens híbridas AG + SA • paralelização • avaliação em bases reais.

Obrigado!

Isabela M. Andrade Júlia A. da Rocha Otávio de Oliveira Pedro B. Luccas

DCC — UNIFAL-MG
